Knowledge Transfer (KT) Document

AI-Powered PDF RAG Agent

Project Name	KT RAG Agent
Purpose	Knowledge Transfer and document-based question answering
Backend	Python + FastAPI
RAG Framework	LangChain
LLM	Google Gemini
Embeddings	Google Gemini Embeddings
Vector Store	FAISS
PDF Parser	PyPDFLoader
API Layer	LangServe
Deployment	Render Web Service

This document is intentionally structured as a realistic project KT guide. It can be used as the source PDF for the RAG Agent and for testing retrieval, page references, technical explanations, and out-of-scope responses.

1. Project Overview

KT RAG Agent is a document-based Knowledge Transfer assistant that allows developers and team members to ask questions about project documentation. The system parses a KT PDF, divides its content into smaller chunks, creates vector embeddings, stores them in a FAISS vector index, retrieves the most relevant chunks for a question, and uses a Google Gemini language model to generate a concise answer.

The system is designed to reduce the time required for a new team member to understand a project. Instead of manually searching a long KT document, the user can ask natural-language questions such as 'How is the application deployed?' or 'What are the main modules?'

2. Project Objectives

• Convert project KT documentation into searchable knowledge.

• Provide natural-language question answering over the KT PDF.

• Retrieve relevant information before generating an answer.

• Reduce hallucination by instructing the agent to use only retrieved KT content.

• Show the source page number whenever possible.

• Provide an HTTP API that can be consumed by a web or mobile frontend.

• Support quick onboarding and knowledge transfer for new developers.

3. Technology Stack

Component	Technology	Purpose
Programming Language	Python 3	Application development
API Framework	FastAPI	REST API and application server
RAG Framework	LangChain	Retrieval and agent orchestration
PDF Parser	PyPDFLoader	Extract text and metadata from PDF pages
Text Splitter	RecursiveCharacterTextSplitter	Create manageable text chunks
Embedding Model	Gemini Embedding	Convert text and queries to vectors
Vector Store	FAISS	Similarity search over document embeddings
LLM	Google Gemini	Generate final natural-language responses
API Integration	LangServe	Expose the RAG chain through HTTP endpoints
Deployment	Render	Host the FastAPI service

4. System Architecture

The application follows a Retrieval-Augmented Generation architecture. The PDF is processed during application startup. Its text is split into chunks, embedded into vectors, and added to FAISS. When a user asks a question, the same embedding model represents the query and FAISS returns the four most similar chunks. The retrieved context is supplied to the RAG agent, which asks Gemini to produce the answer.

• KT PDF → PDF parsing

• Parsed pages → text chunking

• Text chunks → Gemini embeddings

• Embeddings → FAISS vector store

• User question → similarity search

• Top relevant chunks → RAG agent

• RAG agent → Gemini response

• Response → LangServe /agent/invoke endpoint

5. PDF Processing

5.1 PDF Loading

PyPDFLoader reads the KT PDF page by page. Each page is represented as a LangChain Document containing page text and metadata. Page metadata is later used to report the source page in an answer.

5.2 Chunking

The current reference configuration uses a chunk size of 1000 characters and an overlap of 150 characters. The overlap helps preserve context between neighboring chunks.

5.3 Embeddings

Gemini Embeddings transform each text chunk into a numerical vector. The user's question is embedded using the same model so that semantic similarity can be calculated.

6. FAISS Vector Store

FAISS is used for local vector similarity search. The implementation creates an IndexFlatL2 index. The embedding dimension is determined by embedding a test query. The PDF chunks are then added to the FAISS-backed LangChain vector store.

The retrieval function requests the top four similar chunks using similarity_search(query, k=4). The returned chunks include their page number and content.

7. RAG Agent Behavior

The RAG agent is instructed to retrieve information from the KT document before answering document-related questions. It must not use outside knowledge for KT questions and must not invent information.

If the requested information is absent from the document, the expected response is: **I don't know based on the provided KT document.**

8. API Endpoints

Endpoint	Method	Purpose
/	GET	Health/status message for the application
/agent/invoke	POST	Submit a question to the RAG agent
/agent	GET/POST support	LangServe route for interacting with the chain

9. Example Request and Response

Example request: What vector database is used by the KT RAG Agent?

Expected answer: The project uses FAISS as the vector store for similarity search over the embedded KT document chunks. **Source: Page 3.**

Example request: What are the main components of the RAG pipeline?

Expected answer: The main components are PDF parsing, text chunking, embeddings, FAISS retrieval, the RAG agent, and the Gemini language model. **Source: Page 4.**

10. Setup and Installation

• Install Python 3 and create a virtual environment.

• Install dependencies from requirements.txt.

• Set the GOOGLE_API_KEY environment variable.

• Place the KT_Document.pdf file in the same project directory as main.py.

• Start the application with uvicorn main:app --host 0.0.0.0 --port 8000.

• Open the root endpoint to verify that the service is running.

11. Deployment on Render

• Push main.py, requirements.txt, and KT_Document.pdf to the project repository.

• Create a Render Web Service connected to the repository.

• Use 'pip install -r requirements.txt' as the build command.

• Use 'uvicorn main:app --host 0.0.0.0 --port $PORT' as the start command.

• Configure GOOGLE_API_KEY as a Render environment variable.

• Deploy the latest commit and inspect the application logs.

• Verify the root endpoint and then test the /agent/invoke endpoint.

12. Configuration and Environment Variables

Variable	Required	Description
GOOGLE_API_KEY	Yes	Google API credential used for Gemini models and embeddings
PORT	No	Port supplied by Render; defaults to 8000 locally

13. Troubleshooting

PDF file not found

Check that the file is named KT_Document.pdf and is located beside main.py. On Render, do not use a local /content/ path.

GOOGLE_API_KEY is missing

Set GOOGLE_API_KEY in the local environment or in Render Environment Variables. Never commit the actual API key to GitHub.

FAISS import error

Verify that the source code uses exactly 'from langchain_community.vectorstores import FAISS' and that faiss-cpu is present in requirements.txt.

Incorrect or empty answers

Check that the PDF contains extractable text, that the retrieval function returns relevant chunks, and that the agent is instructed to use the retrieval tool.

Render deployment failure

Review the build and runtime logs. Confirm that requirements.txt contains all required packages and that the start command uses $PORT.

14. Security Considerations

• Never store GOOGLE_API_KEY directly in source code.

• Do not commit confidential company KT documents to a public repository.

• Use a private repository for sensitive project documents.

• Restrict access to the deployed API if the KT information is confidential.

• Avoid logging API keys or sensitive document content.

15. Limitations

• The system can only answer accurately when the requested information exists in the source PDF.

• Scanned PDFs without an OCR text layer may require OCR before parsing.

• FAISS data is built during application startup in this reference design.

• Large PDFs can increase startup time and embedding cost.

• The system is not a replacement for human review of critical technical decisions.

16. Frequently Asked Questions

Q: What is RAG?

Retrieval-Augmented Generation combines document retrieval with a language model so the answer can be grounded in retrieved source content.

Q: Why is FAISS used?

FAISS provides efficient vector similarity search over the embedded document chunks.

Q: Why is chunk overlap used?

Overlap helps preserve context that might otherwise be split between two neighboring chunks.

Q: Why are page numbers stored?

Page metadata helps users trace an answer back to the source document.

Q: What happens when information is missing?

The agent should return the defined fallback response instead of inventing an answer.

Q: Can the PDF be replaced?

Yes. Replace KT_Document.pdf with another compatible KT PDF and rebuild/restart the application.

Q: Can a frontend be added?

Yes. A web UI can call the LangServe endpoint and display the agent response.

Q: Where is the API key stored?

It should be stored as an environment variable, such as GOOGLE_API_KEY.

17. Testing Questions for the RAG Agent

Use these questions after deploying the application. The answers should be grounded in this PDF.

1. What is the purpose of the KT RAG Agent?
2. What technologies are used in the project?
3. Which framework is used to build the API?
4. Which library is used to parse the PDF?
5. What is the chunk size used by the text splitter?
6. What is the chunk overlap?
7. Which embedding model is used?
8. Which vector store is used?
9. How many documents are retrieved for a query?
10. What does the retrieval function do?
11. Which LLM generates the final response?
12. What should the agent say if the answer is not in the PDF?
13. What is the purpose of LangServe?
14. What is the API endpoint for invoking the agent?
15. What environment variable stores the Google API key?
16. What is the Render build command?
17. What is the Render start command?
18. What should you check when the PDF file is not found?
19. Why should a confidential KT document not be stored in a public GitHub repository?
20. What are the main limitations of this RAG system?

18. Out-of-Scope Test Questions

These questions are useful for testing whether the agent follows the rule that it should not invent information. The expected response for information not present in the PDF is: **I don't know based on the provided KT document.**

- What is the weather today?

- Who is the current Prime Minister of India?
- Write a recipe for pizza.
- What is the latest version of Python?
- Who founded Google?

19. KT Handover Checklist

- Project overview understood
- Architecture understood
- Technology stack reviewed
- PDF parsing flow reviewed
- Embedding and vector search flow reviewed
- RAG agent behavior understood
- API endpoints tested
- Environment variables configured
- Deployment process tested
- Troubleshooting steps reviewed
- Security considerations reviewed

20. Summary

The KT RAG Agent demonstrates how Retrieval-Augmented Generation can turn a project knowledge document into an interactive Knowledge Transfer assistant. The system parses the KT PDF, chunks the content, creates Gemini embeddings, indexes the vectors with FAISS, retrieves relevant context, and uses Gemini to produce a grounded response. FastAPI and LangServe expose the application as an API, while Render provides a deployment environment.

End of KT Document